JAVA PROGRAM REPORT

Phone Number Letter Combinations Using Recursion

1. Objective / Aim

To generate all possible letter combinations represented by a string of digits from 2 to 9, using the standard telephone keypad mapping and recursion.

2. System Requirements

Hardware: Standard PC/laptop, keyboard, monitor, and sufficient memory/storage.

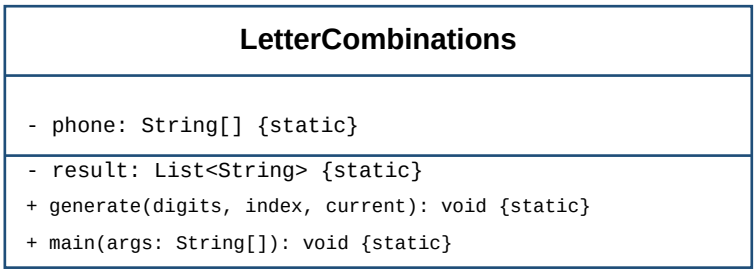
Software: Windows/Linux/macOS, Java Development Kit (JDK), and a Java IDE or command prompt/terminal.

3. Algorithm / Logic Description

1. Read the digit string from the user.
2. If the string is empty, display an empty list.
3. Verify that every character is between 2 and 9; otherwise display a validation message.
4. Map each digit to its corresponding letters using the phone array.
5. Recursively append each possible letter to the current combination.
6. When all digits have been processed, add the completed combination to the result list.
7. Display all generated combinations and close the Scanner.

4. UML Class Diagram

The class uses a keypad mapping array, a list for the generated combinations, a recursive generate method, and main for input validation and output.



5. Source Code

```
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class LetterCombinations {
    static String[] phone = {
        "", "", "abc", "def", "ghi", "jkl",
        "mno", "pqrs", "tuv", "wxyz"
    };

    static List<String> result = new ArrayList<>();

    static void generate(String digits, int index, String current) {
        // Store the combination when all digits are processed
        if (index == digits.length()) {
            result.add(current);
            return;
        }

        int digit = digits.charAt(index) - '0';
        String letters = phone[digit];

        // Try every letter mapped to the current digit
        for (int i = 0; i < letters.length(); i++) {
            generate(digits, index + 1,
                     current + letters.charAt(i));
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
```

```
        System.out.print("Enter digits: ");
        String digits = sc.nextLine();

        if (digits.isEmpty()) {
            System.out.println("Output: []");
        } else {
            boolean valid = true;

            for (int i = 0; i < digits.length(); i++) {
                if (digits.charAt(i) < '2' ||
                    digits.charAt(i) > '9') {
                    valid = false;
                    break;
                }
            }

            if (valid) {
                result.clear();
                generate(digits, 0, "");
                System.out.println("Output: " + result);
            } else {
                System.out.println(
                    "Enter digits from 2 to 9 only."
                );
            }
        }

        sc.close();
    }
}
```

6. Compilation and Execution Commands

```
javac LetterCombinations.java
java LetterCombinations
```

7. Console Output Screenshot

Representative console output from the compiled program:

```
C:\Java-Projects> javac LetterCombinations.java
C:\Java-Projects> java LetterCombinations
Enter digits: 23
Output: [ad, ae, af, bd, be, bf, cd, ce, cf]
C:\Java-Projects> java LetterCombinations
Enter digits: 10
Enter digits from 2 to 9 only.
```

8. Explanation

• **phone array:** Stores the letter groups for digits 2 through 9.

• **Recursion:** Builds combinations by choosing one letter for each digit in order.

• **Base case:** When index equals the length of digits, the completed combination is added to result.

• **ArrayList:** Collects the generated strings in order.

• **Input validation:** Rejects digits outside 2–9; an empty input prints an empty list.

Code check: The program compiles and generates the expected nine combinations for input 23. Empty input prints Output: [], and inputs containing 0, 1, or non-digit characters are rejected. The program's static result list is cleared before generation.